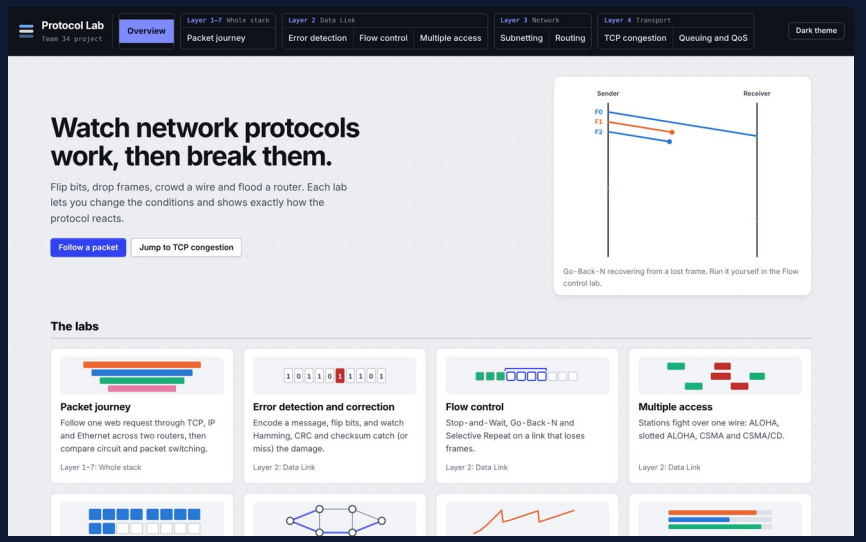
Protocol Lab: how it works, lab by lab

A tour of eight interactive labs that show how computer networks behave.

Savir Ghai · 24BCT0309

Team F1_Team34 · BCSE308P Computer Networks Lab

Faculty: Prof. Sreethar S



What it is, and why I built it

The problem

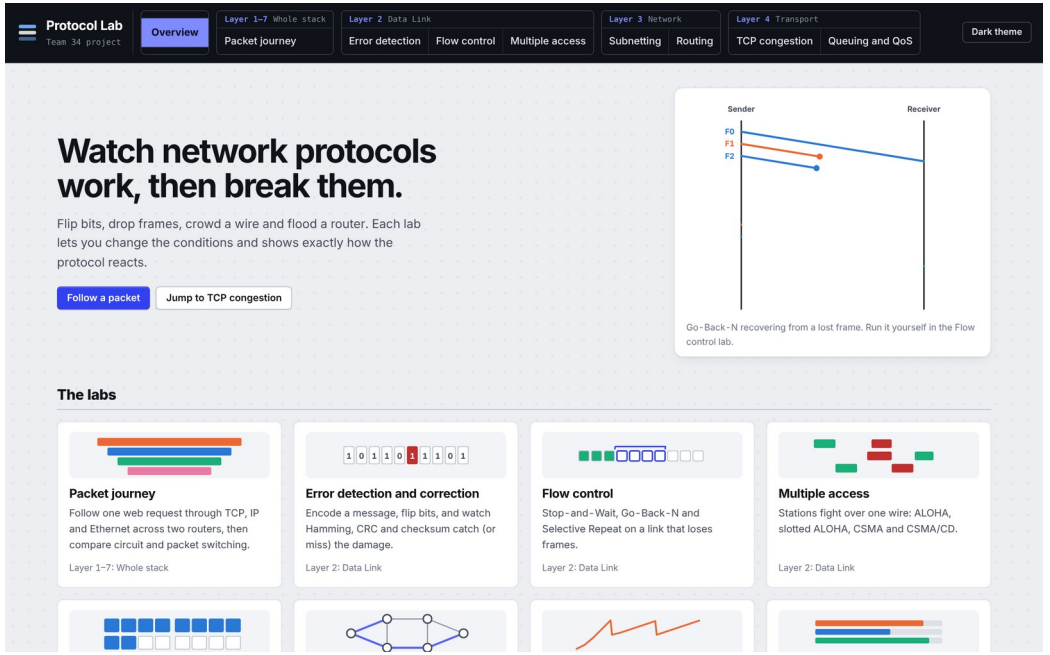
Networking is taught with pictures. A picture shows one case and cannot answer “what if the link gets worse?”

The idea

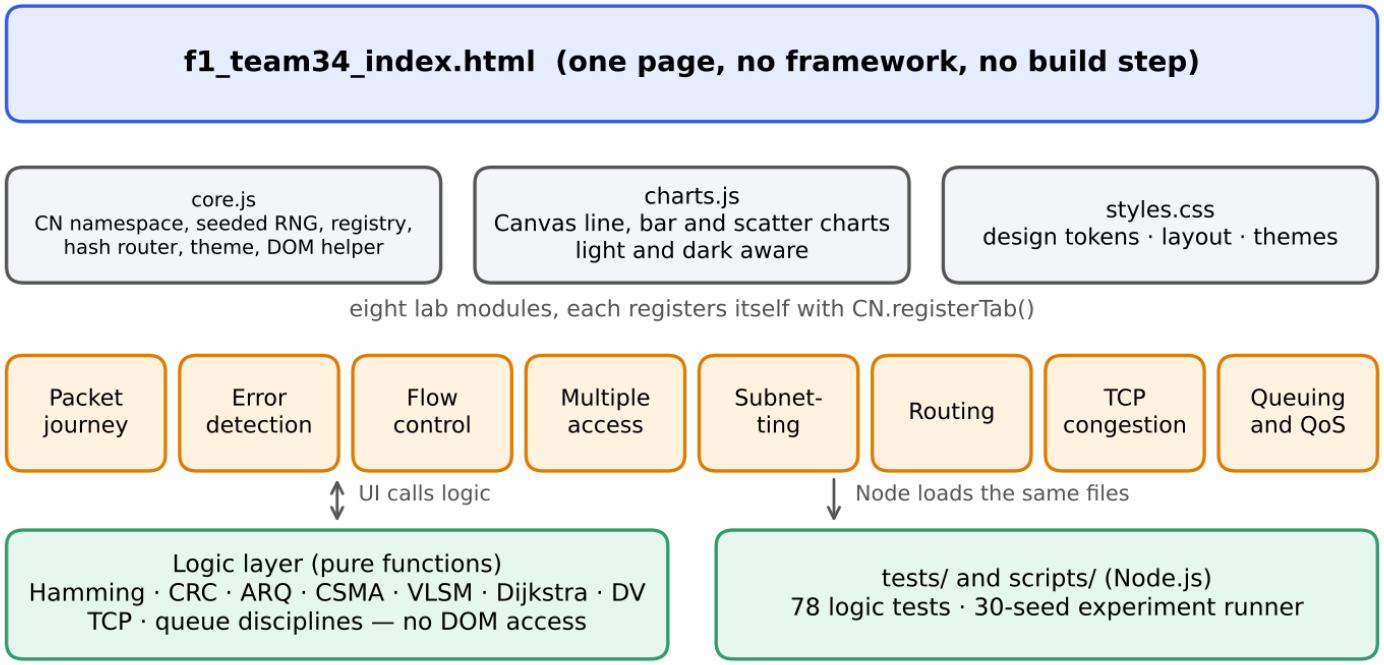
Eight small labs where you change the conditions, press play and watch what the protocol does. Then break something on purpose.

The promise

Open one file and it works. The results are checked by 78 tests, and the same settings always give the same answer.



How it is built, in plain words



Plain web code

HTML, CSS and JavaScript. No framework, no install, no server.

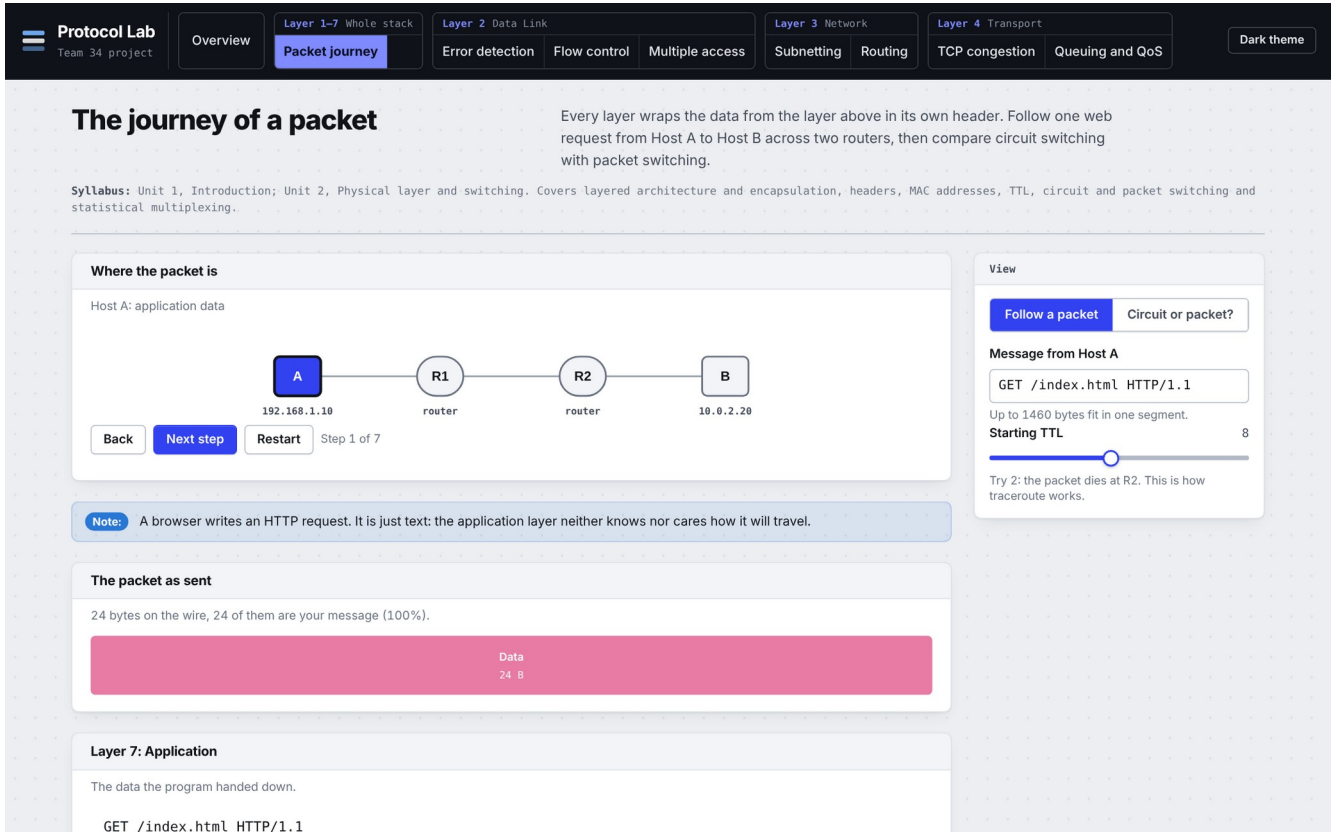
Logic kept apart from the screen

The calculations are plain functions, so the same code runs in Node for testing.

Tested and repeatable

78 tests check known answers. Seeded random numbers give the same result every time.

1. Packet journey: follow one request



Layers 1–7 · Units 1 and 2

The second view, Circuit or packet?, compares the two ways of sending a message: 188 ms against 93.7 ms in the default example.

What it shows

One web request travelling from Host A to Host B across two routers. You see TCP, IP and Ethernet headers wrapped around the data, one layer at a time.

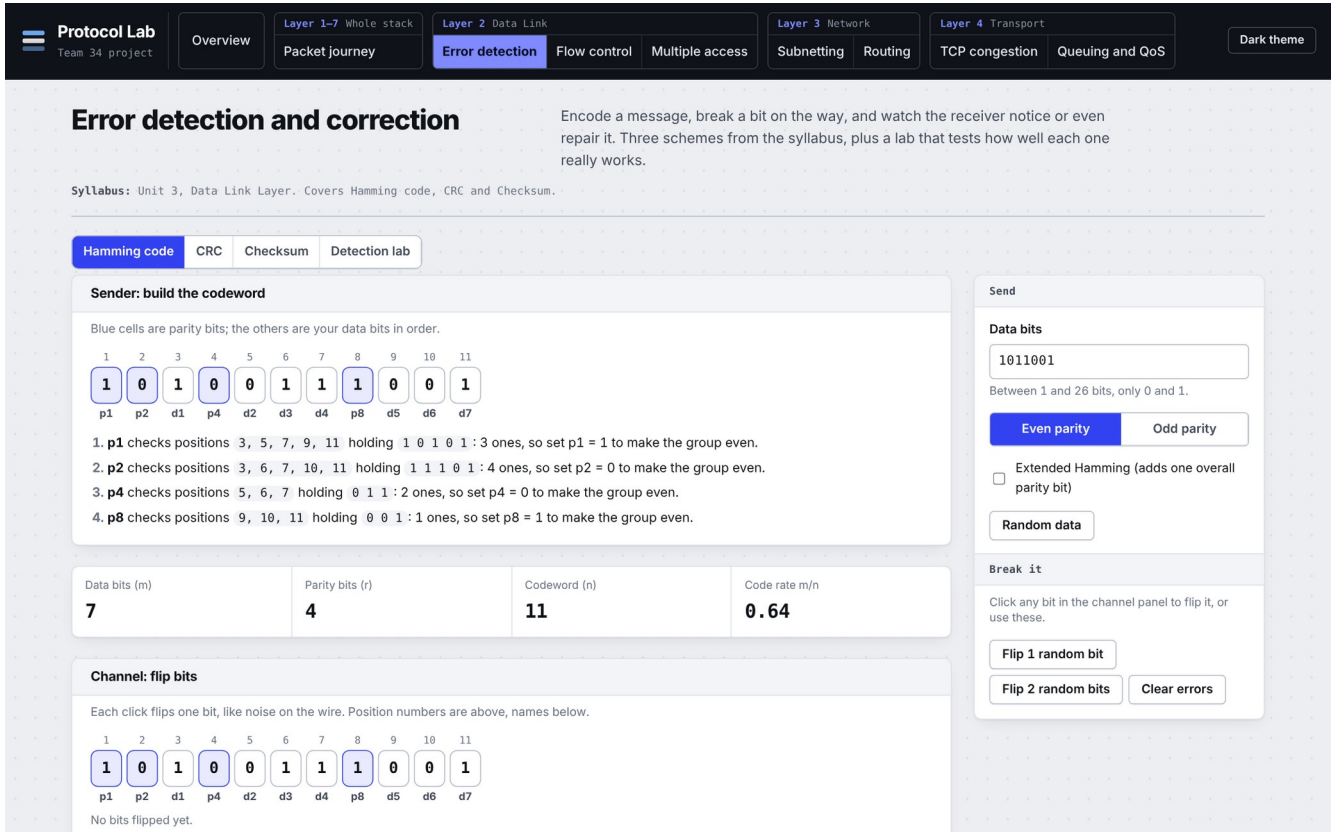
Try this

Click Next step through all 7 steps. Then set the starting TTL to 2 and watch the packet die at router R2.

The takeaway

Headers go on going down and come off going up. Routers rebuild the frame with new MAC addresses, but the IP addresses never change.

2. Error detection and correction



Layer 2 · Data link · Unit 3

Default message: 7 data bits become an 11-bit codeword with 4 parity bits.

What it shows

How a sender adds extra bits so the receiver can notice, or even repair, damage on the wire. It has Hamming code, CRC and checksum, plus a lab that tests all three.

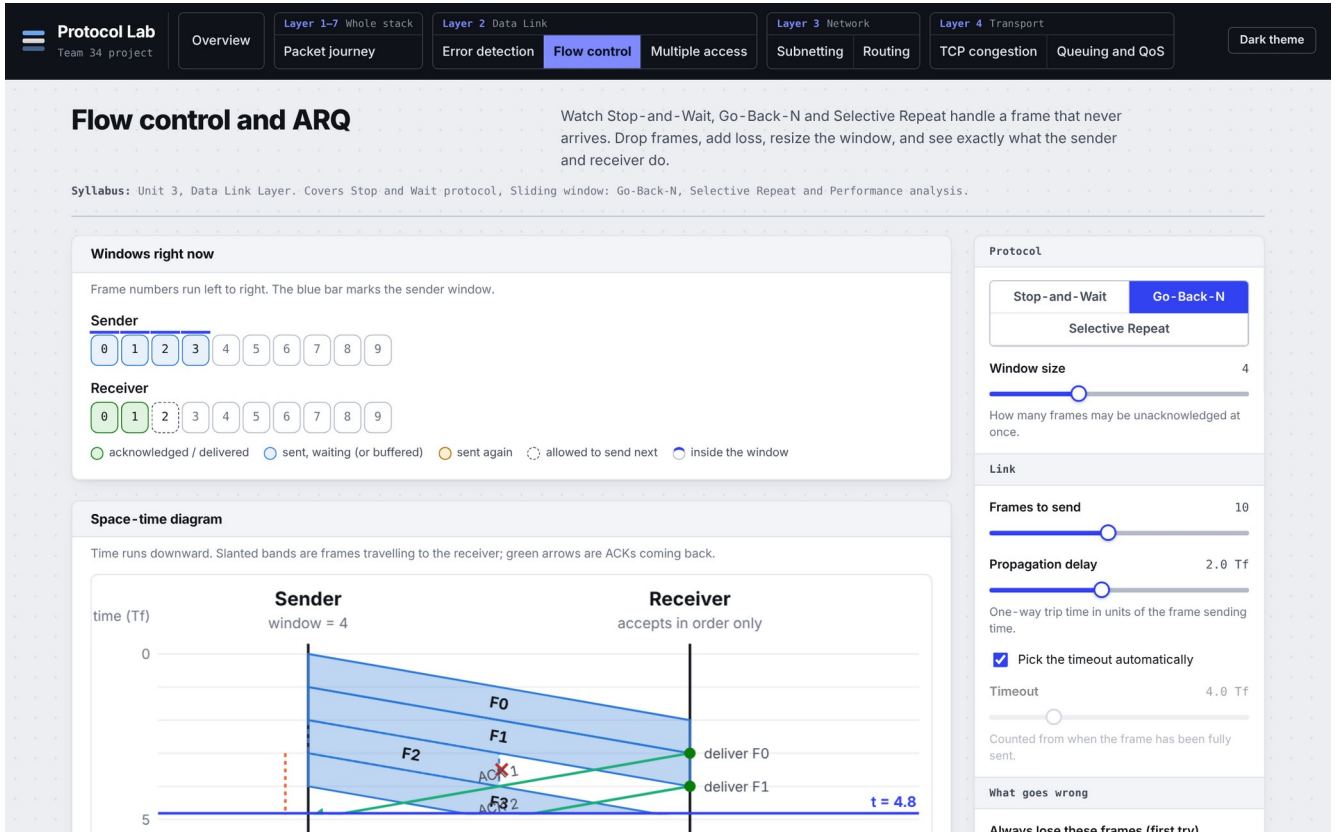
Try this

On the Hamming tab, click Flip 1 random bit and watch it get corrected. Then click Flip 2 random bits.

The takeaway

Extra bits cost space but buy safety. Hamming can fix one flipped bit; with two, plain Hamming gets it wrong, so the extended version detects them.

3. Flow control: keep the sender busy



Layer 2 · Data link · Unit 3

Stop-and-Wait finishes in 55.1 time units, the two sliding-window schemes in 20.1.

What it shows

A sender and a receiver on a link that can lose frames. A time diagram shows every frame and every acknowledgement. You pick Stop-and-Wait, Go-Back-N or Selective Repeat.

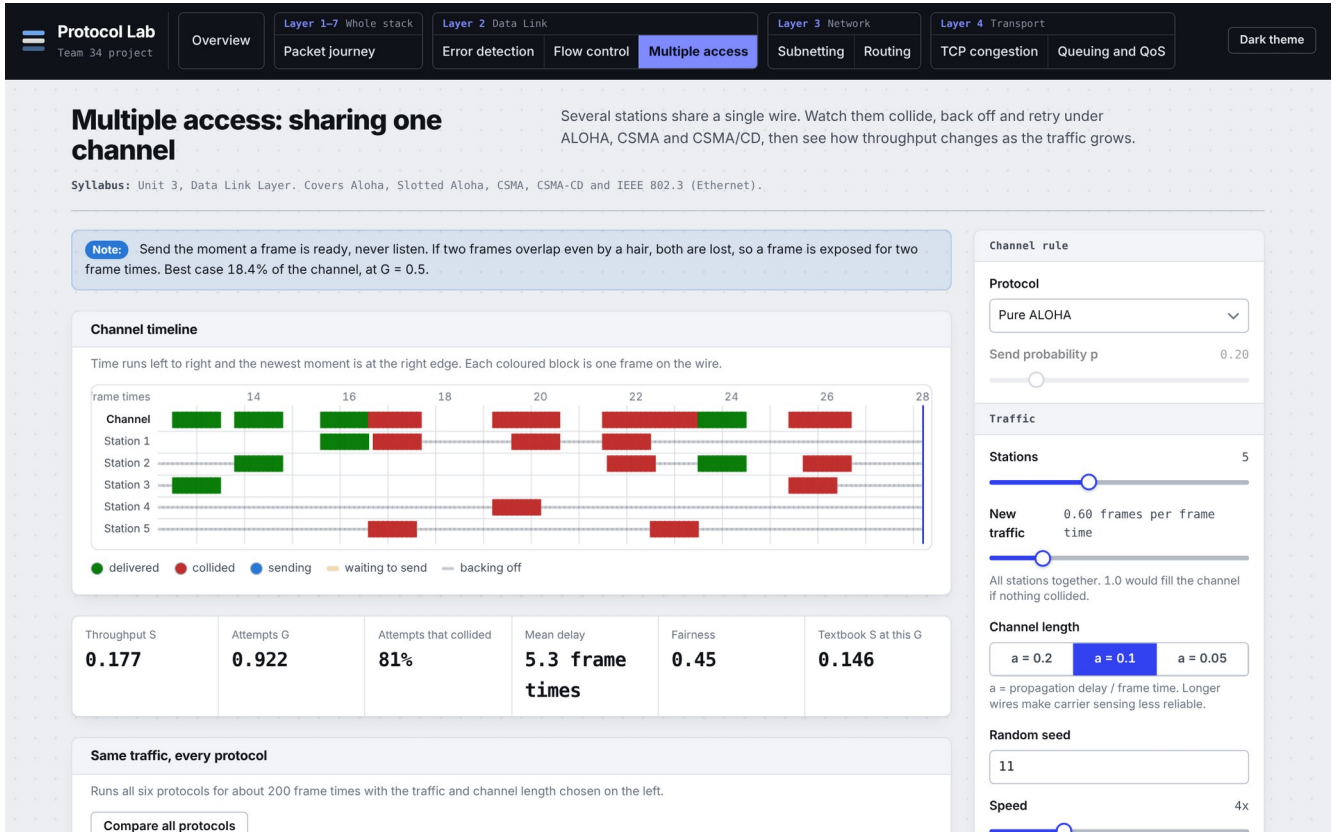
Try this

With the defaults (frame 2 is lost), click Go-Back-N, then Show everything. Then do the same with Selective Repeat.

The takeaway

A window lets the sender keep sending. Go-Back-N resends the whole window after a loss (4 frames here); Selective Repeat resends only the lost one.

4. Multiple access: sharing one wire



Layer 2 · Data link · Unit 3

The a value (0.2, 0.1, 0.05) is how long the wire is compared with a frame. A short wire helps carrier sensing.

What it shows

Many stations that all want to use the same wire. When two talk at once their frames collide. The lab runs pure ALOHA, slotted ALOHA, two kinds of CSMA, and CSMA/CD.

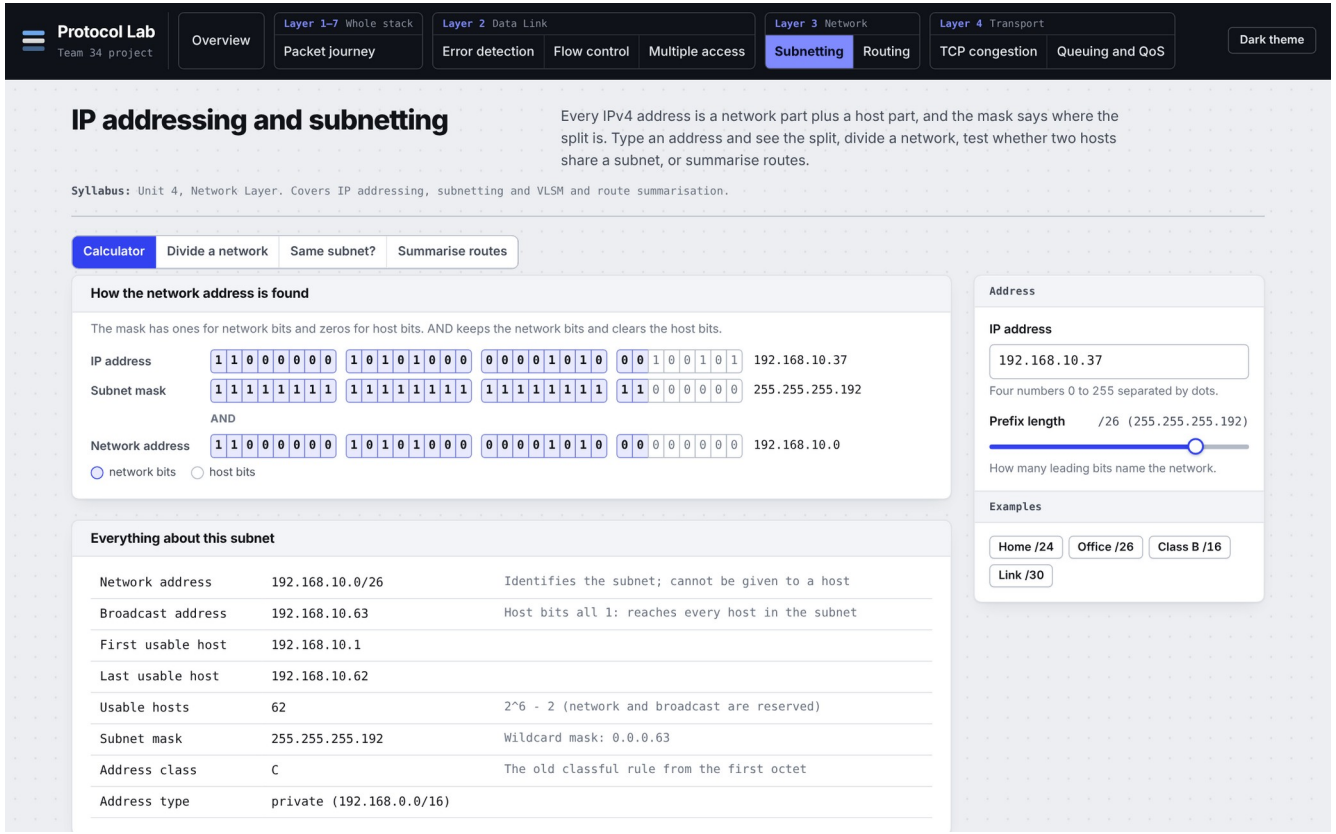
Try this

Click Compare all protocols to see throughput against how busy the network is.

The takeaway

Waiting for your turn beats talking at random. Pure ALOHA tops out around 0.184 of the link, slotted ALOHA around 0.368, and listening first (CSMA) does better still.

5. Subnetting: cutting up an address space



Layer 3 · Network · Unit 4

Dividing for 100, 50, 20 and 2 hosts gives a /25, /26, /27 and /30, using 228 of the 256 addresses.

What it shows

An IP address is a network part plus a host part, and the mask says where the split is. The lab has a calculator, a tool to divide a network, a same-subnet test and route summarisation.

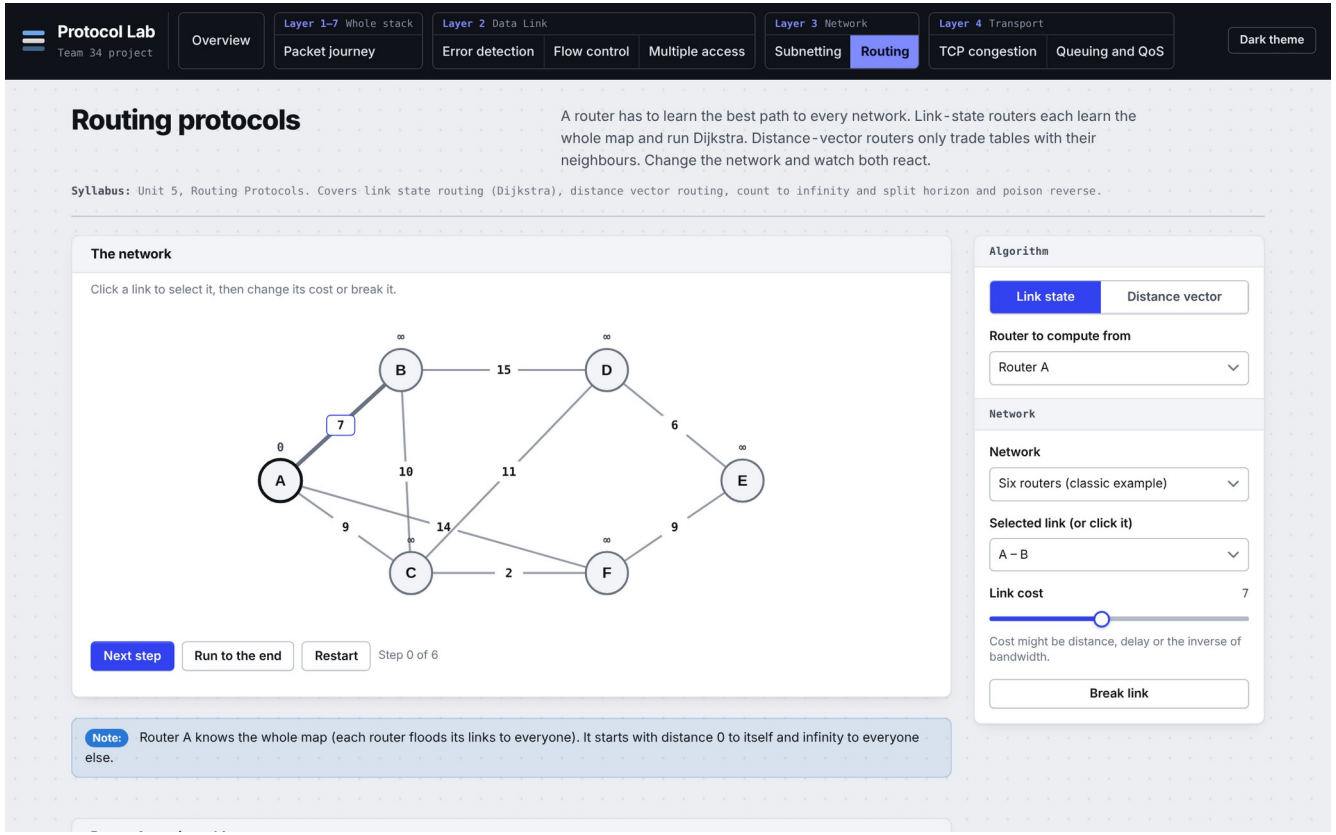
Try this

Type 192.168.10.37 with /26. Then divide 192.168.10.0/24 for 100, 50, 20 and 2 hosts.

The takeaway

The mask is just ones for the network bits and zeros for the host bits. For /26: network 192.168.10.0, broadcast 192.168.10.63, 62 usable hosts.

6. Routing: how routers find the way



Layer 3 · Network · Unit 5

In Link state mode, Next step shows Dijkstra from A: B 7, C 9, F 11, D and E 20.

What it shows

Two ways routers learn the map. Link state: each router knows everything and runs Dijkstra. Distance vector: routers only trade tables with their neighbours.

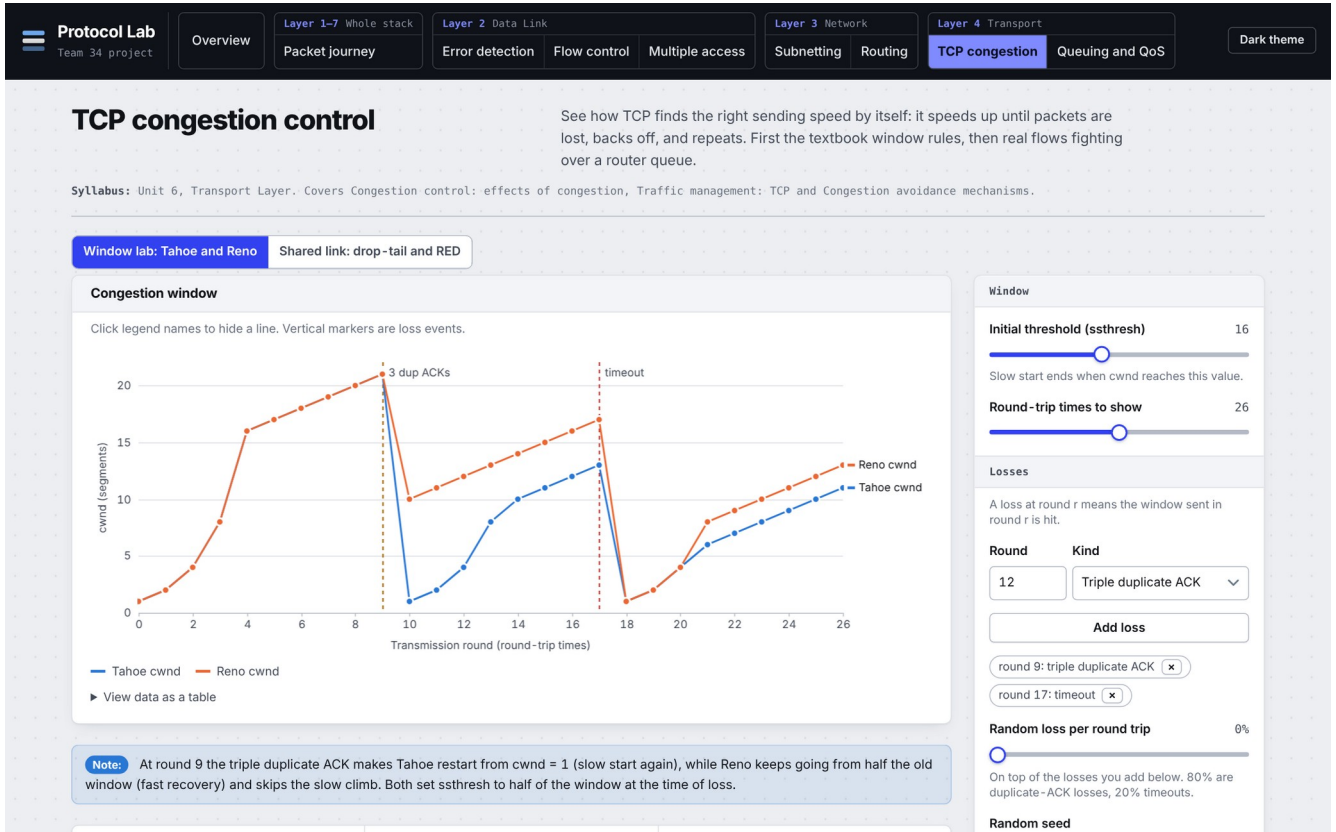
Try this

Distance vector, Three in a line, run until stable. Select link B – C, Break link, run again. Then switch to Split horizon and repeat.

The takeaway

After a break, plain distance vector counts up to infinity (32 here, Round 32 on screen). Split horizon stops routers telling each other a route they learned from each other.

7. TCP congestion: how much to send



Layer 4 · Transport · Unit 6

The other tab, Shared link, has several flows sharing one router with drop-tail or RED queues.

What it shows

How TCP decides its sending speed. The window doubles in slow start, then grows by one per round, and drops when a loss is detected. Tahoe and Reno react differently.

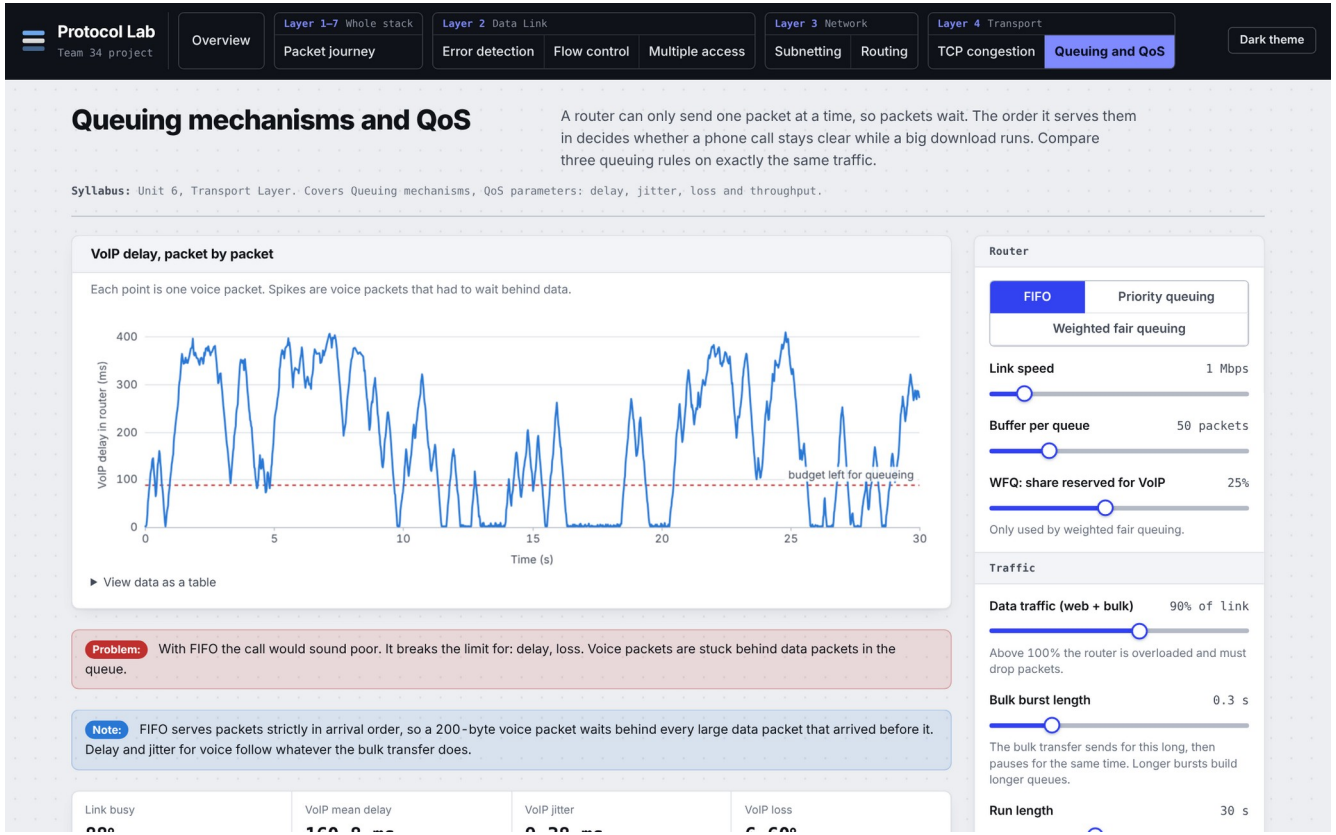
Try this

Open the Window lab with the defaults: a triple duplicate ACK at round 9 and a timeout at round 17. Read the totals under the chart.

The takeaway

On three duplicate ACKs Reno halves its window and Tahoe restarts from 1, so Reno gets more through: 304 segments against 245. On a timeout both restart from 1.

8. Queuing and QoS: protecting a phone call



Layer 4 · Transport · Unit 6

A call counts as good with 95th percentile delay under 150 ms, jitter under 30 ms and loss under 1%.

What it shows

A router with one slow line shared by a phone call, web traffic and a big download. The order the packets leave in decides how good the call sounds.

Try this

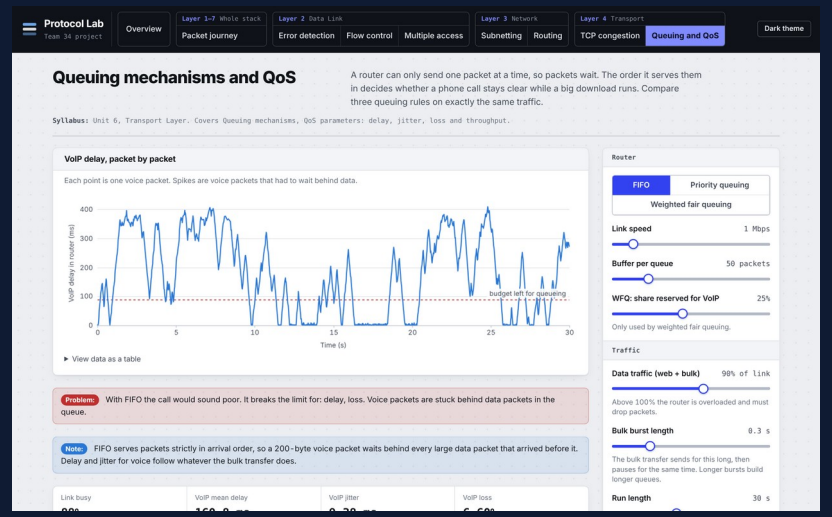
The default is FIFO. Then click Priority queuing, then Weighted fair queuing.

The takeaway

FIFO makes voice wait behind big downloads: 160.8 ms mean delay and 6.60% loss. Priority gives 5.64 ms and WFQ 7.18 ms, both with no loss.

Wrap-up

- Eight labs covering Units 1 to 6, from encapsulation to queuing and QoS.
- Each mechanism is shown next to its simpler baseline, and measured over 30 random seeds in the report.
- Simplified on purpose: it teaches how things behave, not real-world numbers.
- Not covered yet: DNS, HTTP and FTP (Unit 7). A DNS lab is next.




```
git clone https://github.com/savir18/CN.git
open f1_team34_index.html
```